SEC 8-K Product Announcement Analysis

Hieu Nguyen | CAP-5619 | Assignment 2

GitHub: https://github.com/hingsgn/sec-8k-llm-product-analysis

Purpose and observed results

This project structures new product releases reported in SEC Form 8-K filings into the required five-column CSV. The recorded local LLM run analyzed 104 filings from 10 companies and produced 102 reviewed product entries. Retained entries come from 31 filings. Filing dates span 2023-01-31 through 2025-12-10. Filings with no retained products remain in the audit records; they are not padded into the product CSV.

Eligibility covers named product introductions or availability announcements reported in a filing, including releases summarized for its reporting period. A filing's acceptance timestamp is not asserted to be the product's original launch date. This distinction matters for earnings-release exhibits that recap recent introductions.

Data selection and preprocessing

The targeted sample uses Item 2.02 earnings 8-Ks for NVDA, AMD, INTC, CSCO, CRM, ORCL, IBM, HPE, HPQ and ADBE within 2023-2025. Ten filings per issuer plus two older filings each for NVIDIA and AMD give 104 filings. This is an enrichment strategy for finding product updates, rather than a random or market-wide sample.

The collector resolves tickers with company_tickers.json and obtains filing histories from the official SEC Submissions JSON API, including historical continuation files. It replaces the older browse-edgar Atom lookup while retaining SEC as the data source. Filing-detail pages identify authoritative acceptance times and exhibits. The archive contains 245 primary reports and EX-99 attachments. Identified requests share a two-per-second limit, retries and raw-response caching.

BeautifulSoup removes HTML markup, scripts and hidden XBRL metadata. One announcement-rich exhibit is selected per filing, with the primary 8-K as fallback. Event sentences and issuer context form an input passage capped at 10,000 characters. All cleaned source documents are retained; passage filtering and exhibit selection are explicit recall limitations.

Local LLM and named entity recognition

Python queries Qwen2.5-3B-Instruct using llama-cpp-python 0.3.36 and the qwen2.5-3b-instruct-q4_k_m.gguf weights (Q4_K_M quantization). The publisher revision is 7dabda4d13d513e3e842b20f0d435c732f172cbe. Settings are temperature 0, seed 42, an 8,192-token context and up to 900 output tokens. No paid API is required.

The instruction-tuned LLM performs NER by returning organization and product names in schema-constrained JSON. It reads source text as data and excludes financial events, third-party products and ordinary existing-product mentions. Saved records contain the actual raw responses, input passages, token usage, timing and model identifiers. ORGANIZATION and PRODUCT offsets make recognized entity spans auditable.

Grounding, review and output construction

Company names and tickers come from SEC issuer metadata. The filing-detail page's Accepted time is interpreted in America/New_York and exported as an ISO 8601 timestamp with a UTC offset. LLM-proposed product names must map to the source passage; recovery only reconciles case, spacing and trademark variants.

Each retained product requires an actual source sentence containing its name and an announcement or availability cue. Descriptions are concise source extracts or documented source-based review edits. Repeated company/product pairs are deduplicated. Coannounced model groups may remain one source entity instead of being expanded into unsupported names.

The CSV header is exactly company_name, stock_name, filing_time, new_product, product_description. A separate text table uses vertical-bar separators. Provenance, entity offsets, review exclusions and inference logs remain in JSON sidecars so that the required CSV has no extra columns.

Challenges and safeguards

A primary 8-K may contain only financial or legal boilerplate and refer readers to an exhibit. Following EX-99 attachments supplies the product-bearing text. Initial small-model attempts produced omissions and unrelated entities; shorter announcement-focused passages made the final local run practical. Source matching alone can still retain an existing product named in a partnership, benchmark or customer deployment. Documented review decisions remove these cases, generic entities and overlapping references while preserving the original model responses.

The earlier Ollama workflow also encountered a stopped local server and repeated ConnectionRefusedError messages. This submission uses the in-process llama.cpp runtime instead of a localhost server. Its default replay mode regenerates the submission from saved real LLM checkpoints without rerunning inference. Fresh inference remains available with RERUN_LLM=True, and new outputs require a renewed evidence review.

Validation and reproducibility

Validation passes for 102 nonblank product rows, the exact column order, timezone-aware filing timestamps, unique company/product pairs, exact source evidence and NER offsets. Every description is strictly shorter than 180 characters; the observed maximum is 129. Five offline regression tests cover invented names, ordinary existing-product mentions, valid announcements, trademark recovery and the character limit. Checkpoint replay reproduces the CSV byte for byte.

These checks establish formatting and traceability, not independently measured precision or recall. The targeted sample, selected exhibit, sentence filter, passage cap and model omissions can miss valid releases. The project does not claim 100% accuracy. AI assistance supported implementation, documentation and evidence review. Code and required outputs are published at the GitHub URL above; that URL also appears in the prepared CV header.

Primary references

SEC EDGAR APIs: https://www.sec.gov/search-filings/edgar-application-programming-interfaces

SEC developer guidance: https://www.sec.gov/about/developer-resources

Qwen weights: https://huggingface.co/Qwen/Qwen2.5-3B-Instruct-GGUF

Python runtime: https://github.com/abetlen/llama-cpp-python